

Les 1 AI vs ML

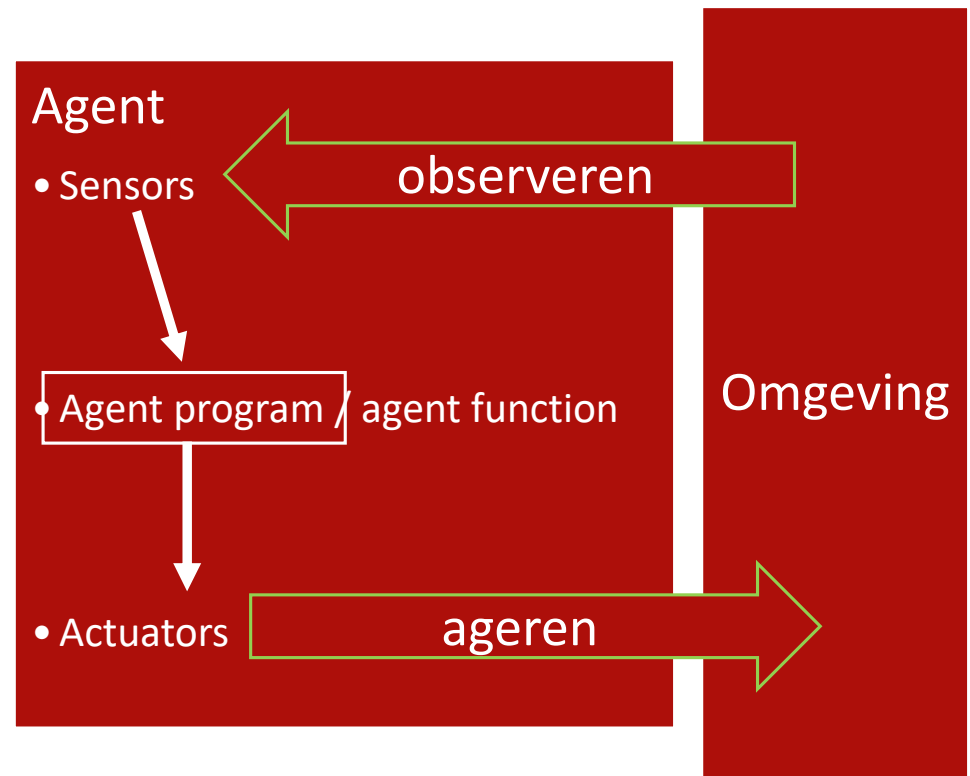
Wat is het verschil ?

Intelligente agents

Een belangrijk denkkader

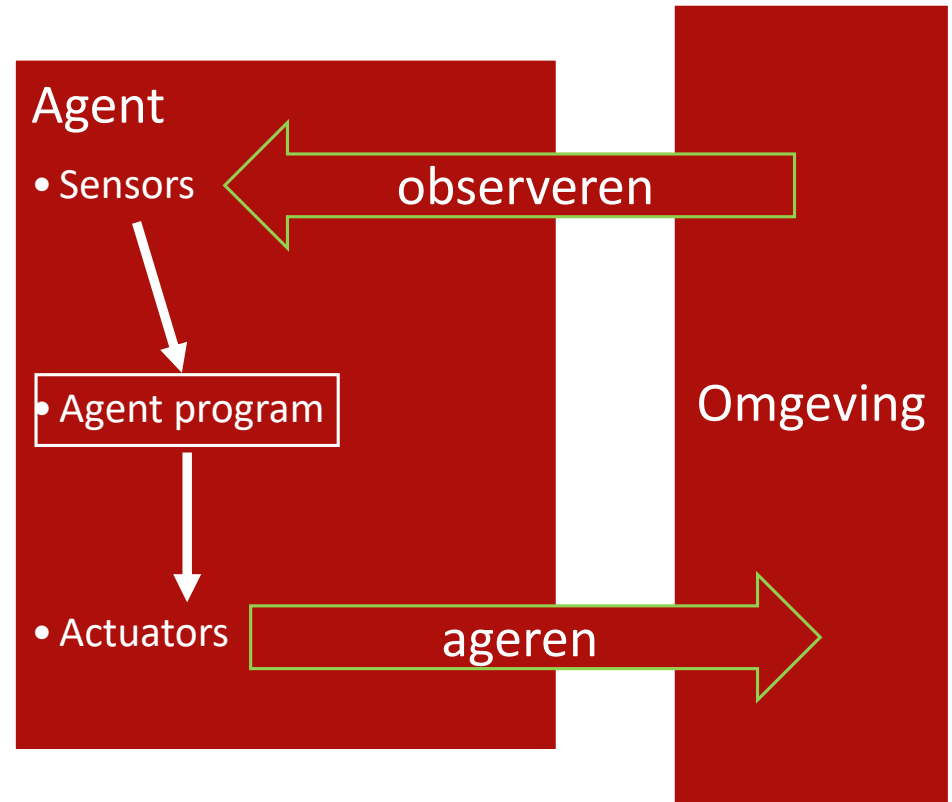
Agents

- Een agent is een systeem dat via **sensors** informatie binnenkrijgt uit zijn **omgeving**. De agent gaat op basis daarvan en interne logica (**agent program**) via **actuators** acties uitvoeren op diezelfde omgeving, in de hoop een bepaalde situatie te verbeteren



Agents

- Zowel het 'observeren' als het 'ageren' zijn dingen waar fouten kunnen optreden:
 - Een sensor kan fout meten
 - Een actuator kan een motor aansturen met als doel vooruit te gaan, maar als die motor een technisch defect heeft, blijf je alsnog staan



Een foutieve sensor – 737 Max



- ‘Flawed information from a single external sensor fed into the system caused it to repeatedly push the planes’ noses down as pilots struggled to keep them in the air before both crashes.’
[bron:
https://en.wikipedia.org/wiki/Boeing_737_MAX]
- Dit systeem had een ‘**single point of failure**’ in de vorm van die sensor. Het leidde tot 2 vliegtuigcrashes, met bijna 350 doden.
- Netflix: “Downfall: the case against Boeing”

Agent function

- Een agent is bepaald door zijn **agent function**. In principe zou je voor elke mogelijk reeks van inputs, kunnen bepalen wat de agent moet doen (een tabel maken) – die tabel kunnen we de state-action tabel noemen
- In de praktijk is de tabel gigantisch, soms zelfs oneindig groot
 - Voor schaken: 10^{150} entries
 - Ter info, aantal atomen in universum: circa 10^{80}
- De agent function is de theoretische werking van een agent

Staat (gras, plaats rechtdoor)	Actie
(niet gemaaid, plaats)	Maaien
(gemaaid, plaats)	Rechtdoor
(gemaaid, geen plaats)	Links
...	

De state-action tabel is dus een soort fata morgana: ziet er fantastisch uit, maar kan in realiteit niet bestaan !

Agent function



Voorbeeld:

- Een **robotgrasmaaier** moet beslissen of hij rechtdoor, links, rechts, achteruit gaat, stopt, de maaier aan of afzet. De omgeving van de robot – je tuin – geeft informatie: als de maaier van het gras rijdt, zou het systeem links of rechts of achteruit moeten gaan. De robot past de omgeving ook aan: idealiter is op het einde heel je tuin gemaaid

Wat is AI in dit framework ?

- De taak van AI is om een **agent program** te ontwerpen dat de agent function implementeert
 - Dit is dus **concrete code**
- Deze is ontworpen voor een specifieke set actuatoren en sensors – de **agent architecture**
 - Het heeft geen zin om aan geavanceerd beeldherkenning en object-detectie te doen als er geen camera of camerabeeldenstroom is
- Deze cursus gaat over agent programs, hoe ze te ontwerpen en onderling te vergelijken

Rationele Agents

- Wat maakt een agent 'rationeel' ?
 - Voor elke geobserveerde input, moet de agent handelen zodat zijn 'doelfunctie' wordt gemaximaliseerd
 - Dus we hebben een doelfunctie nodig die bepaalt hoe goed een agent bezig is

Voorbeeld:

- Het doel van de grasmaaier is (zo snel mogelijk) de hele tuin maaien. Een doelfunctie (**performance measure**) zou kunnen zijn: het % reeds gemaaid gras.
- Een grasmaaier die dus in rondjes draait en bepaalde stukken van de tuin niet maait, is niet rationeel.

Rationele Agents - voorwaarden

- Wat maakt een agent 'rationeel' ?
 - Bovendien houdt een rationele agent rekening met zijn geobserveerde data, en eventuele **ingebouwde kennis**
 - Echter, de agent kan onmogelijk alles weten: een autonoom voertuig kan niet weten dat er een niet-geconnecteerde auto achter de hoek met 100 km/u het kruispunt nadert. Rationeel $\neq$ alleswetend!
 - In onze definitie gaan we er ook vanuit dat de agent op basis van de historiek aan inputs in staat is om te **leren**
 - Bovendien moet een rationele agent dit **autonoom** kunnen doen

Voorbeeld:

- Hier kunnen we een onderscheid maken tussen 2 soorten robotmaaiers:
 - Robotmaaiers die de vorm van jouw tuin kennen voordat ze aan hun rondje beginnen
 - Robotmaaiers die nog geen idee hebben hoe de tuin eruit ziet. Deze moeten eerst de vorm van de tuin leren. De tweede keer gras maaien moet hier een stuk sneller zijn.

Task environment

- Een agent kan enkel worden geëvalueerd op basis van de taak waarvoor hij is ontworpen
 - Dit zijn in essentie de **‘problemen’**, de agents zijn de **‘oplossingen’**
- Een task environment: **PEAS (POAS in Nederlands)**
 - P: Performance measure
 - E/O: Environment / Omgeving
 - A: actuators, wat het systeem kan aansturen
 - S: sensors, wat het systeem kan ‘voelen / zien / percipiëren’

Task environment

- Voor de robotmaaier

Agent type	Performance	Environment	Actuators	Sensors
Robotmaaier	Gras maaien, veilig tegen snij-incidenten, minimaal energieverbruik, minimale menselijke tussenkomst	Grasveld, bomen, planten, dieren (dynamisch), weersomstandigheden	Sturen, banden rechtdoor, achteruit, maaier op / af, waarschuwingslichtje,	Hoogte gras, weerstand op wielen, navigatie tov basisstation, batterij autonomie, korte-nabijheid sensor, temperatuur, vochtigheid gras

Task environment

- Voor een automatische scan van een paspoort op het gemeenteloket

Agent type	Performance	Environment	Actuators	Sensors
OCR Scanner (optical character recognition)	Foto automatisch omzetten in beeld, minimize extra correctie keyboard input	Linux file omgeving; inputfolder Jpgfiles, PNG-files, max 25mb Outputfolder max 10 gb HD, systeem met 4GB RAM, Keyboard input	Files wegschrijven naar disk, resultaat weergeven op scherm	Camera, keyboard input registratie

Wat voor soort problemen lossen we op ?

- Observeerbaarheid
 - Perfect vs imperfect
- Single vs multi
 - Multi: competitive vs cooperative
- Deterministisch vs stochastisch
- Episodisch vs Sequentieel
- Statisch vs Dynamisch
- Discreet vs Continu

Spelletjes van allerlei soort

Fully observable vs partially observable

- Volledig observeerbaar:
 - Schaak
 - Dammen
 - Go (chinees bordspel)
 - Monopoly
 - Mens-erger-je-niet
- Dit noemen ze ook wel **perfect information games**



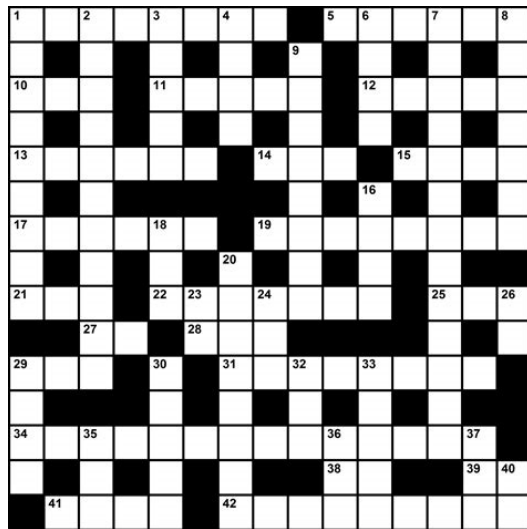
- Partieel observeerbaar
 - Poker
 - Patience
- Dit noemen ze ook **imperfect information games**



Single vs Multi

- **Single**

- Kruiswoordraadsel
- Patience



- **Multi**

- 2-player:
 - Schaken, dammen, go
 - Dit zijn typisch **competitieve** problemen
- Multi-agent
 - Zelfrijdende auto's
 - **Cooperatief** systeem
 - Scrabble
 - Monopoly

Single vs Multi

- **Deterministisch**

- Schaken, dammen, go
 - In volledig observeerbare, deterministische spellen moet de speler geen rekening houden met *onzekerheid*

- **Stochastisch**

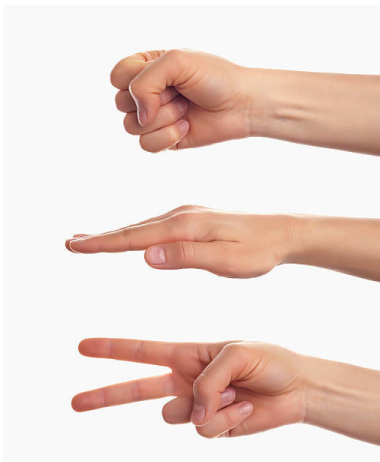
- Mens-erger-je-niet
- Monopoly
- (alles met een dobbelsteen)



Episodisch vs sequentieel

- **Episodisch**

- Schaar-steen-papier
- Roulette



- **Sequentieel**

- Lottotrekking (ballen worden niet teruggelegd)



Statisch vs dynamisch

- **Statisch**

- Schaak (zonder klok)
- Kruiswoordraadsel
- Heel veel bordspellen
- Puzzels

- **Dynamisch**

- Veel computerspellen, zoals race-spellen, schietspellen, ...
- **Semi-dynamisch:** indien enkel de performantie-score wijzigt met de tijd
 - Schaak met een klok
 - Escape games



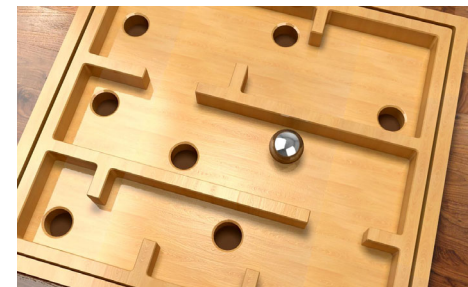
Discreet vs continu

- **Discreet**

- Schaak, dammen, go
- Bijna alle bordspellen

- **Continu**

- Typisch voor échte problemen
 - Autonomous driving
 - Camera-detecting algorithms
 - Strikt genomen is dit discreet, maar in de praktijk continu. Er zijn namelijk op een typische camera minstens 256^3 kleurtinten -> 16 miljoen kleuren, voor het blote oog is dat continu
- Of voor spellen met echte stukken, zoals dit knikkerbord:



Agent Programs

Agents - Soorten

- De uitdaging voor AI is het schrijven van een programma dat een rationele agent oplevert zonder heel de state-action tabel te definiëren.

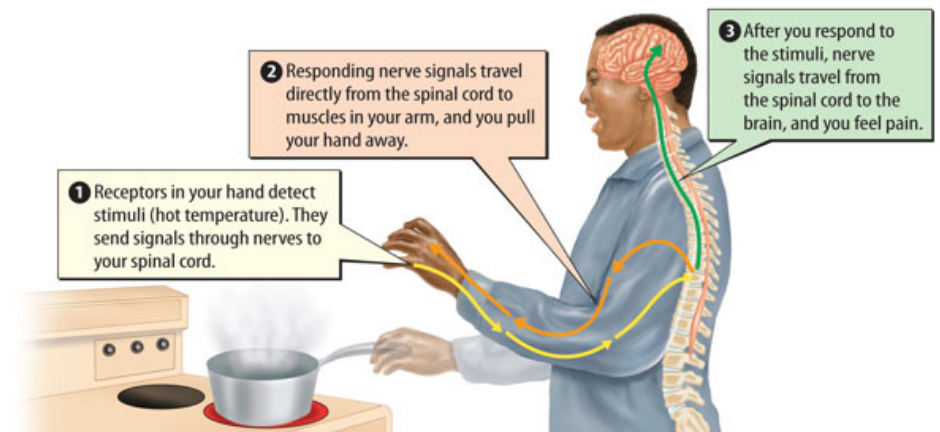
We geven 3 soorten AI systemen, die eigenlijk de basis vormen van alle onderliggende systemen:

- Simple Reflex agents
- Model based Reflex agent
- Goal based agents; Utility agents

Simple Reflex agents

- Ageren op basis van de huidige sensor input
 - **Geen rekening houden met oudere input**
 - Deze hebben dus een set regels:
conditie → actie
IF → THEN
- Ze zijn eenvoudig, maar het is duidelijk dat ze qua intelligentie beperkt zijn

- Mensen hebben zelf zulke ingebouwde “reflexen”:
 - Hand terugtrekken bij verbranding



Simple Reflex agents

- Deze agents komen snel in de problemen in situaties die niet volledig observeerbaar zijn (imperfect information)
- We kijken naar een sphex wesp, die niet in staat is om de geschiedenis van het verleden om te zetten in slimmer gedrag:
- https://www.youtube.com/watch?v=YNvi_j2z96w



Model-based Reflex agents

- Deze agents houden in hun geheugen bij wat ze momenteel niet kunnen waarnemen
 - Heeft dus een **interne state**
 - Kan dus veranderingen over de tijd vaststellen in de omgeving
 - ‘Weet’ wat het effect is van haar acties – dit noemen we een **transition model**
 - ‘Weet’ hoe veranderingen van de omgeving worden gereflecteert door **sensormetingen – sensor model**
 - Bij elke nieuwe input gaat dit model zijn interne **state** moeten **updaten**
- De robotgrasmaaier heeft een soort kaart van jouw tuin, waar hij al wel en niet is geweest
- Het transition model van de grasmaaier weet dat als hij de linkermotor harder laat draaien dan de rechtermotor, de robot naar rechts stuurt

Model-based Reflex agents

- Deze agents krijgen het moeilijk wanneer er veel mogelijkheden zijn.
- **Zoekproblemen** hebben typisch meer intelligentie nodig, omdat er niet genoeg tijd is om heel de zoekruimte te checken
 - Wanneer 'brute force' niet meer voldoende is
- Voorbeeld
 - Google Maps kiest de optimale route tussen Antwerpen en Parijs. Het heeft geen tijd om alle mogelijke wegen te checken. Het gebruikt onderweg het doel 'zo snel mogelijk van Antwerpen naar Parijs' geraken om bijvoorbeeld een weg via Amsterdam uit te sluiten

Goal based agents en utility agents

- Deze agents hebben steeds een doel wat aangeeft of nieuwe situaties wenselijk zijn of niet.
- Dit is geen “reflex” meer, het probeert iets te doen wat de state dichterbij de uiteindelijke uitkomst brengt
- Het is meer flexibel.
- Soms wordt er een onderscheid gemaakt tussen
 - Goal Based – enkel het einddoel is gekend
 - Utility Based – hier heeft het systeem intern een **utility functie**, die het wil maximaliseren – op een tussenpositie kan er ook worden aangegeven of dit beter of slechter is dan de vorige
- Een goal/doel voor de zoekfunctie van Antwerpen naar Parijs is om een pad te vinden dat start in Antwerpen en eindigt in Parijs.
 - Een pad is dan een rij steden met een weg ertussen
- Een **utility functie** is dan heel concreet:
 - $-d(\{\text{huidige stad}\}, \{\text{nieuwe stad}\})$

Dit wil je zo groot mogelijk hebben, dus een zo klein mogelijk afstand (d van distance)

Sorteren

Even een opfrissing

Wat is sorteren ?

- Onder sorteren verstaan we het ordenen van een lijst, waarbij er een gekende manier is om 2 items in de lijst onderling te vergelijken que grootte:
 - $<$ of $>$
 - Alfabetisch
 - Chronologisch
- Sorteren werd al gedaan in de jaren vijftig, maar vrij recent zijn er nog nieuwe algorithms gepubliceerd én zeer populair (Timsort, 2002)

Complexiteit van Algoritmes

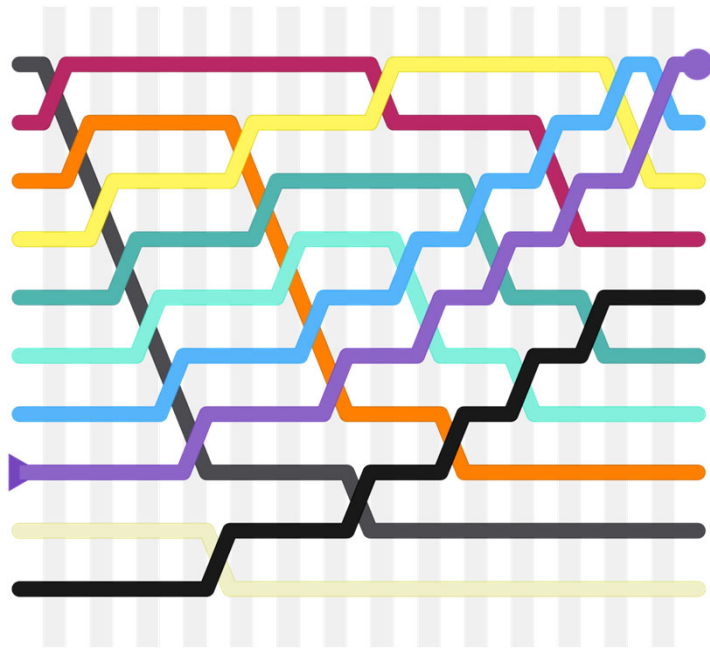
- Hoe kunnen we vergelijken of het ene algoritme sneller is dan het andere ?
 - Een algoritme wordt sowieso trager als we er meer data aan geven
 - Afhankelijk van het systeem waarop het algoritme wordt gehost kan het ook sneller / trager gaan
 - Veel hangt ook af of het probleem *parallelliseerbaar* is

Big O Notation

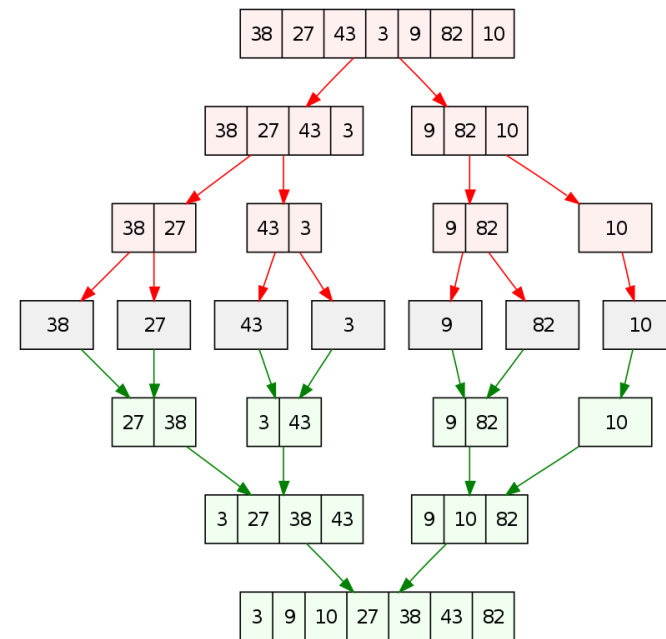
- Dit dient om algoritmes onderling te vergelijken
 - Efficiëntie in tijd
 - Efficiëntie in geheugengebruik ('space complexity')
- $O(1)$
 - Het programma wordt niet trager bij meer input
- $O(n)$
 - Het programma wordt dubbel zo traag bij dubbel zoveel input
- $O(n^2)$: het programma wordt 4 (2^2) keer zo traag bij dubbel zoveel input.

Voorbeeld: 2 sorteeralgoritmes

Bubble Sort



Merge Sort



Voorbeeld: 2 sorteeralgoritmes



demo

Verschillende sorteeralgoritmes

- Insertion sort (gaan we bekijken tijdens het labo)
- Quicksort
 - Zoals de naam het zegt, erg snel
- Tree sort
 - Specifiek voor lijsten waar items aan worden toegevoegd

- Op wikipedia vind je een goede overzichtstabel:

Comparison sorts							
Name	Best	Average	Worst	Memory	Stable	Method	Other notes
Quicksort	$n \log n$	$n \log n$	n^2	$\log n$	No	Partitioning	Quicksort is usually done in-place with $O(\log n)$ stack space. ^{[5][6]}
Merge sort	$n \log n$	$n \log n$	$n \log n$	n	Yes	Merging	Highly parallelizable (up to $O(\log n)$ using the Three Hungarians' Algorithm). ^[7]
In-place merge sort	—	—	$n \log^2 n$	1	Yes	Merging	Can be implemented as a stable sort based on stable in-place merging. ^[8]
Introsort	$n \log n$	$n \log n$	$n \log n$	$\log n$	No	Partitioning & Selection	Used in several STL implementations.
Heapsort	$n \log n$	$n \log n$	$n \log n$	1	No	Selection	
Insertion sort	n	n^2	n^2	1	Yes	Insertion	$O(n + d)$, in the worst case over sequences that have d inversions.
Block sort	n	$n \log n$	$n \log n$	1	Yes	Insertion & Merging	Combine a block-based $O(n)$ in-place merge algorithm ^[9] with a bottom-up merge sort.
Timsort	n	$n \log n$	$n \log n$	n	Yes	Insertion & Merging	Makes $n-1$ comparisons when the data is already sorted or reverse sorted.
Selection sort	n^2	n^2	n^2	1	No	Selection	Stable with $O(n)$ extra space, when using linked lists, or when made as a variant of Insertion Sort instead of swapping the two items. ^[10]
Cubesort	n	$n \log n$	$n \log n$	n	Yes	Insertion	Makes $n-1$ comparisons when the data is already sorted or reverse sorted.
Shellsort	$n \log n$	$n^{4/3}$	$n^{3/2}$	1	No	Insertion	Small code size.
Bubble sort	n	n^2	n^2	1	Yes	Exchanging	Tiny code size.
Exchange sort	n^2	n^2	n^2	1	No	Exchanging	Tiny code size.
Tree sort	$n \log n$	$n \log n$	$n \log n$ (balanced)	n	Yes	Insertion	When using a self-balancing binary search tree.
Cycle sort	n^2	n^2	n^2	1	No	Selection	In-place with theoretically optimal number of writes.

History slide of the day

De inceptie van AI – 1943-1956

- Jaren 50: Oplossen van simpele spelletjes en puzzels - pionierstijdperk
- 1958: J. McCarthy creëert programmeertaal **Lisp**: eerste taal waarmee je nieuwe regels kon toevoegen zonder het te herprogrammeren
- Microworlds:
 - SAINT: integralen oplossen (1963)
 - ANALOGY: vormproblemen uit IQ tests (1967)
 - **Block world**
 - Soort heel simplistische versie van Minecraft
 - SHRDLU, een eerste chatbot (Terry Winograd)

An early natural language understanding system: SHRDLU (Winograd, 1972)

